

# Operaciones de Machine Learning

## Proyecto Final

### Nivel 4: Automatización, decisión de reentrenamiento y despliegue GitOps

Pontificia Universidad Javeriana

Cristian Javier Diaz Alvarez

20 de mayo de 2026

## 1. Descripción

Este proyecto final evalúa la capacidad del estudiante para diseñar, implementar, desplegar y explicar un sistema de *Machine Learning Operations* (MLOps) con un flujo automatizado de recolección de datos, validación, procesamiento, entrenamiento, versionamiento, despliegue, inferencia y observabilidad. El objetivo no es únicamente entrenar un modelo, sino construir un sistema que tome decisiones técnicas justificadas durante el ciclo de vida del modelo.

El sistema debe consumir datos desde una API externa, almacenar la información cruda, procesarla, evaluar si existe evidencia suficiente para reentrenar un modelo, ejecutar el entrenamiento cuando corresponda, registrar los experimentos en MLflow, comparar el nuevo modelo contra el modelo actualmente productivo y actualizar la fuente de verdad del modelo solamente cuando el nuevo modelo demuestre mejor desempeño según las métricas definidas. La API de inferencia debe consumir el modelo productivo desde MLflow sin requerir cambios de código ni redespiegue de la aplicación.

El proyecto debe ser desarrollado bajo una visión de producción. Por lo tanto, se espera que los estudiantes consideren escenarios de fallo, trazabilidad, reproducibilidad, versionamiento, pruebas de carga, observabilidad, límites de recursos, despliegue en Kubernetes y gestión declarativa mediante GitOps.

### 1.1. Objetivos de aprendizaje

Al finalizar el proyecto, el estudiante deberá ser capaz de:

- Diseñar un flujo MLOps automatizado que integre recolección de datos, validación, procesamiento, entrenamiento, registro de modelos e inferencia.
- Implementar DAGs en [Airflow](#) con múltiples tareas y bifurcaciones de decisión.
- Separar correctamente datos crudos y datos procesados en bases de datos distintas o esquemas claramente diferenciados.
- Registrar experimentos, métricas, parámetros, artefactos y modelos en [MLflow](#).
- Definir criterios técnicos para decidir si se debe entrenar un nuevo modelo o no.
- Comparar un modelo candidato contra el modelo actualmente productivo y justificar si debe promoverse o rechazarse.
- Exponer el modelo productivo mediante [FastAPI](#), tomando MLflow como fuente de verdad.
- Construir una interfaz en [Streamlit](#) para inferencia y visualización del historial del proceso de entrenamiento.
- Automatizar la construcción y publicación de imágenes de contenedor mediante [GitHub Actions](#).
- Desplegar todos los componentes en Kubernetes usando manifiestos versionados y sincronizados mediante [Argo CD](#).

- Instrumentar al menos la API de inferencia con métricas observables en Prometheus y Grafana.
- Ejecutar pruebas de carga con Locust y analizar sus efectos en la API mediante dashboards de observabilidad.

## 1.2. Condiciones generales del proyecto

Todos los componentes del sistema deben ejecutarse en contenedores. Las imágenes propias del proyecto deben construirse mediante workflows de GitHub Actions y publicarse en DockerHub. No se permite construir imágenes manualmente en la máquina de despliegue. Kubernetes debe consumir las imágenes desde DockerHub u otro registro de contenedores equivalente autorizado por el docente.

Todos los componentes deben desplegarse en un clúster de Kubernetes local o proporcionado para el curso. Los manifiestos, archivos Helm o configuraciones equivalentes deben estar versionados en el repositorio Git. Argo CD debe ser el mecanismo encargado de sincronizar los manifiestos con el clúster, adoptando una visión GitOps.

Cada contenedor debe declarar explícitamente `resources.requests` y `resources.limits`. La ausencia de límites y mínimos de recursos será considerada una deficiencia técnica, ya que impide evaluar el comportamiento del sistema bajo condiciones controladas.

## 2. Descripción del dataset

Los datos corresponden a registros de propiedades inmobiliarias provenientes de un escenario inspirado en listados de bienes raíces. El problema de aprendizaje consiste en estimar el precio de una propiedad a partir de sus características estructurales, geográficas y comerciales.

|    | Variable       | Descripción                                                                                                               |
|----|----------------|---------------------------------------------------------------------------------------------------------------------------|
| 1  | brokered_by    | Agencia o corredor codificado categóricamente.                                                                            |
| 2  | status         | Estado de la vivienda, por ejemplo, lista para la venta o lista para construir.                                           |
| 3  | price          | Precio objetivo de la propiedad. Corresponde al precio de cotización actual o al precio de venta reciente cuando aplique. |
| 4  | bed            | Número de habitaciones.                                                                                                   |
| 5  | bath           | Número de baños.                                                                                                          |
| 6  | acre_lot       | Tamaño del terreno en acres.                                                                                              |
| 7  | street         | Dirección codificada categóricamente.                                                                                     |
| 8  | city           | Ciudad donde se encuentra la propiedad.                                                                                   |
| 9  | state          | Estado o región.                                                                                                          |
| 10 | zip_code       | Código postal.                                                                                                            |
| 11 | house_size     | Área habitable de la vivienda, expresada en pies cuadrados.                                                               |
| 12 | prev_sold_date | Fecha de venta anterior, cuando exista.                                                                                   |

Tabla 1: Descripción de variables del dataset.

El dataset no será entregado completo desde el inicio. La información será obtenida por partes mediante una API externa. Cada petición representa un nuevo lote de datos disponible en el sistema. Por lo tanto, los estudiantes deben construir un proceso incremental que evalúe cada lote, actualice las tablas correspondientes y determine si el nuevo lote justifica un nuevo entrenamiento.

### 2.1. Carga del dataset desde API

Los datos serán obtenidos a través de una API externa expuesta como imagen de DockerHub. La URL definitiva de la imagen será informada por el docente:

### DockerHub image: cristiandiaz13/mlops-puj:data-api-pf-v1

La API entregará subconjuntos de datos distintos de acuerdo con el grupo asignado (pueden usar 1 para el grupo sin problema) y el avance de las peticiones. Cada ejecución del DAG debe consumir un lote, almacenarlo en la base de datos cruda y continuar con el proceso completo de validación, transformación y decisión de entrenamiento. No se permite descargar todos los lotes de una vez para luego entrenar un único modelo final.

Cada petición debe tratarse como un nuevo evento de datos en el sistema. Por esta razón, el DAG debe registrar el identificador del lote, fecha de ejecución, cantidad de registros, esquema recibido, clases o categorías nuevas detectadas, validaciones realizadas, decisión tomada y resultado final.

No se recomienda usar la interfaz gráfica de Swagger para inspeccionar grandes volúmenes de datos. La API debe consumirse desde código, preferiblemente desde una tarea de Airflow diseñada para manejar errores, reintentos y validaciones de respuesta. Para instanciar la imagen pueden usar esta referencia: `docker run -rm -p 8000:80 cristiandiaz13/mlops-puj:data-api-pf-v1`

## 3. Arquitectura

La arquitectura de referencia integra herramientas de orquestación, almacenamiento, seguimiento de experimentos, registro de modelos, inferencia, visualización, CI/CD, observabilidad y despliegue declarativo. El sistema debe reflejar un flujo completo de MLOps y no una colección aislada de servicios.

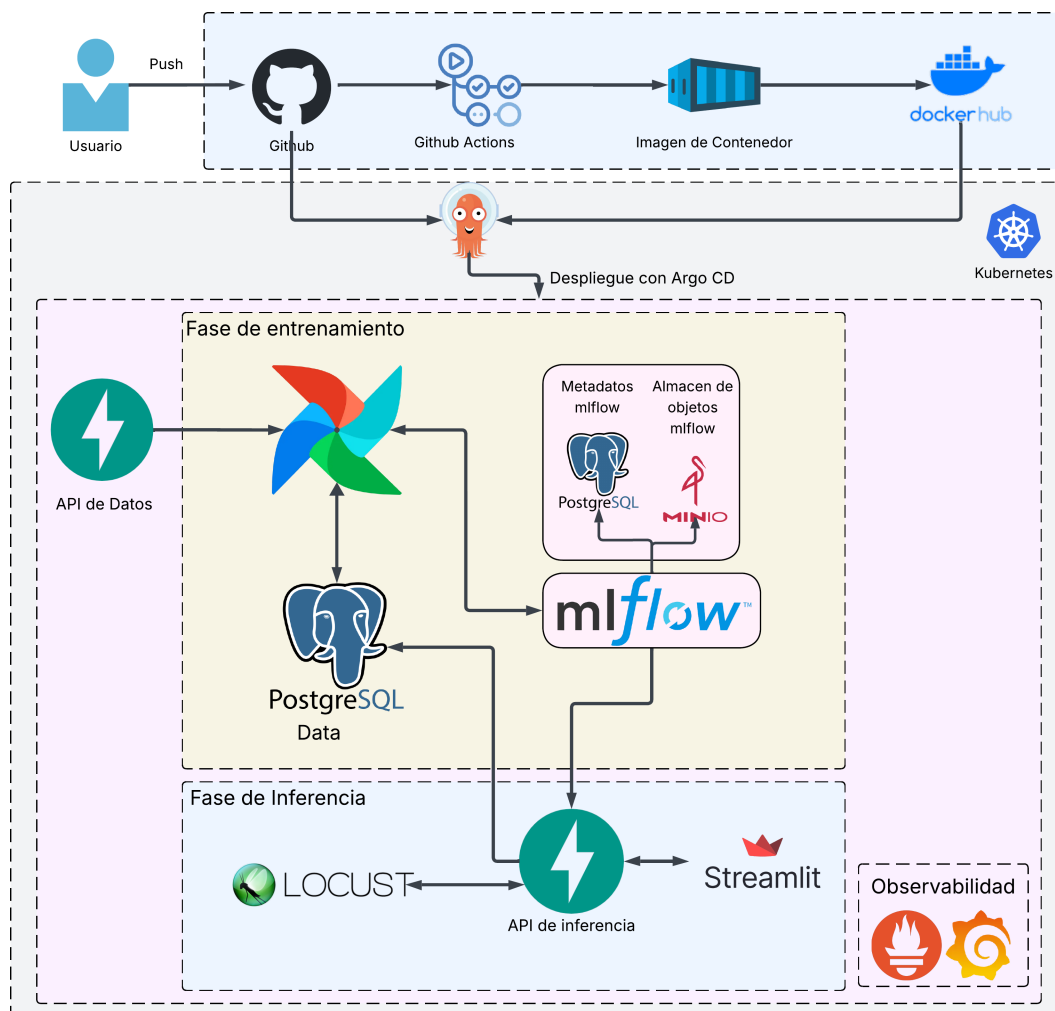


Figura 1: Arquitectura de referencia del proyecto final.

El flujo mínimo esperado es el siguiente:

1. GitHub almacena el código fuente, DAGs, Dockerfiles, manifiestos de Kubernetes, configuraciones de observabilidad y documentación.
2. GitHub Actions construye y publica imágenes de contenedor en DockerHub.
3. Argo CD sincroniza los manifiestos versionados en Git con el clúster de Kubernetes.
4. Airflow consume datos desde la API externa, valida el lote, almacena datos crudos y genera datos procesados.
5. El DAG decide si se debe entrenar un nuevo modelo con base en criterios técnicos definidos por el equipo.
6. Si se entrena, el modelo candidato se registra en MLflow junto con métricas, parámetros, artefactos, validaciones y explicación de la decisión.
7. El modelo candidato se compara contra el modelo productivo actual.
8. Si el candidato mejora el desempeño según las reglas definidas, se actualiza su alias, tag o etiqueta productiva en MLflow.
9. FastAPI consulta MLflow como fuente de verdad y debe poder actualizar el modelo cargado sin reconstruir ni redespargar la imagen.
10. Streamlit permite probar inferencias y consultar el historial de decisiones de entrenamiento y promoción de modelos.
11. Prometheus recolecta métricas de la API de inferencia y Grafana permite visualizar el comportamiento del sistema, especialmente durante pruebas de carga con Locust.

## 4. Requerimientos funcionales

### 4.1. RF1. Recolección incremental de datos

El sistema debe consumir la API de datos desde Airflow. Cada ejecución del DAG debe solicitar un nuevo lote de datos, almacenarlo en RAW DATA y registrar metadatos de la ejecución. El sistema debe evitar reprocesar accidentalmente un mismo lote, salvo que el estudiante implemente una estrategia explícita de reprocesamiento controlado.

### 4.2. RF2. Persistencia separada de datos crudos y procesados

La información recibida desde la API debe almacenarse sin modificaciones en una base o esquema RAW DATA. Posteriormente, el pipeline debe generar una versión procesada en CLEAN DATA. La separación entre ambos niveles es obligatoria, ya que permite trazabilidad, auditoría y reproducibilidad.

### 4.3. RF3. Validación de datos y control de cambios

Antes de entrenar, el DAG debe ejecutar validaciones sobre el lote recibido y sobre el conjunto acumulado disponible. Como mínimo, se espera evaluar:

- Cambios de esquema: columnas nuevas, columnas faltantes o cambios de tipo de dato.
- Calidad de datos: nulos, duplicados, rangos inválidos, valores extremos y consistencia del target.
- Volumen de datos: cantidad de registros nuevos y proporción frente al histórico.
- Cambios en distribución: comparación entre el lote nuevo y la distribución histórica o de referencia.
- Aparición de nuevas categorías o clases en variables categóricas.
- Suficiencia de datos para entrenamiento y validación.

El sistema no debe fallar automáticamente ante una nueva categoría. Debe aplicar una estrategia robusta, por ejemplo, codificadores capaces de manejar categorías desconocidas, actualización controlada del espacio categórico, uso de `handle_unknown`, agrupación en categoría *other* o lógica equivalente técnicamente justificada.

#### 4.4. RF4. Decisión automática de entrenamiento

El DAG debe incluir una bifurcación explícita que decida si se entrena o no se entrena un nuevo modelo. Esta decisión no puede depender únicamente de periodicidad ni de la cantidad bruta de lotes recolectados. Debe sustentarse en reglas técnicas.

Ejemplos de criterios válidos:

- Se detectó cambio significativo en la distribución de una o varias variables relevantes.
- Se detectaron nuevas categorías con frecuencia suficiente para afectar el modelo.
- El lote nuevo aumenta el volumen acumulado en un porcentaje mínimo definido.
- Se observa degradación del desempeño del modelo actual sobre una muestra reciente etiquetada.
- El esquema cambió, pero el pipeline pudo adaptarse y validar correctamente los datos.
- No se entrena porque el lote es demasiado pequeño, incompleto, inválido o no aporta variación relevante.

Toda decisión debe quedar registrada en una tabla de auditoría del proceso y debe ser visible desde Streamlit.

#### 4.5. RF5. Entrenamiento y registro en MLflow

Cuando el DAG determine que sí debe entrenar, debe ejecutar el pipeline de entrenamiento, registrar el experimento en MLflow y guardar como mínimo:

- Identificador del lote o conjunto de lotes usados.
- Versión del código o commit asociado.
- Parámetros del modelo y del preprocesamiento.
- Métricas de entrenamiento, validación y prueba.
- Artefactos relevantes, como gráficos de desempeño, matriz de errores o reportes de interpretabilidad.
- Modelo serializado usando la estructura de MLflow Models.
- Razón por la cual se decidió entrenar.

#### 4.6. RF6. Comparación contra el modelo productivo

Entrenar un modelo no implica promoverlo automáticamente. Después del entrenamiento, el DAG debe comparar el modelo candidato contra el modelo productivo actual registrado en MLflow. La comparación debe usar las métricas definidas por el equipo. Para un problema de regresión de precios, se recomienda usar métricas como MAE, RMSE, MAPE o  $R^2$ , justificando cuál de ellas será prioritaria.

El modelo candidato solo debe promoverse si mejora al modelo productivo bajo una regla explícita, por ejemplo:

*Promover si el MAE disminuye al menos 3 % y el RMSE no empeora más de 1 %.*

Si el modelo candidato no supera al productivo, debe quedar registrado como experimento, pero no debe reemplazar el modelo usado por la API.

## 4.7. RF7. Actualización del modelo en la API sin redesplicue

FastAPI debe usar MLflow como fuente de verdad del modelo productivo. No se permite quemar rutas locales, nombres de archivos fijos o versiones específicas dentro del código de inferencia.

La API debe implementar un mecanismo de actualización del modelo sin redesplicue la aplicación. Se aceptan estrategias como:

- Endpoint administrativo protegido para recargar el modelo desde MLflow.
- Verificación periódica del alias, tag o stage productivo en MLflow.
- Recarga bajo demanda cuando la versión productiva registrada cambie.

La estrategia debe evitar que una petición de inferencia quede en estado inconsistente durante la recarga. El estudiante debe explicar cómo maneja concurrencia, errores de descarga y fallback al modelo previamente cargado.

## 4.8. RF8. Registro de inferencias

Cada petición realizada a la API de inferencia debe dejar registro en RAW DATA o en una tabla de eventos de inferencia conectada al dominio RAW. El registro debe incluir, como mínimo, fecha y hora, datos de entrada, predicción, versión del modelo usada, estado de la petición y errores si existen. Estos datos deben poder usarse posteriormente como insumo para monitoreo o futuros entrenamientos.

## 4.9. RF9. Interfaz de usuario en Streamlit

La interfaz de Streamlit debe tener dos secciones obligatorias:

1. **Inferencia:** formulario para ingresar datos de una propiedad, consumir la API FastAPI, visualizar la predicción y mostrar la versión del modelo utilizada.
2. **Historial de entrenamiento y despliegue:** vista que muestre el resultado de cada lote procesado. Debe indicar si se entrenó o no, la razón de la decisión, si el modelo fue promovido o rechazado, el cambio de desempeño frente al modelo productivo y los identificadores de MLflow asociados.

Ejemplo de historial esperado:

- **Batch 1:** entrenó porque fue la línea base inicial. Se promovió porque no existía modelo productivo previo.
- **Batch 2:** no entrenó porque el lote fue pequeño y no presentó cambio de distribución relevante.
- **Batch 3:** entrenó por cambio en la distribución de `house_size` y aparición de nuevas ciudades. No se promovió porque el MAE fue 4 % peor que el modelo productivo.
- **Batch 4:** entrenó por acumulación suficiente de datos recientes y mejora de cobertura categórica. Se promovió porque redujo el MAE en 6 % y mantuvo estable el RMSE.

La cantidad máxima de lotes disponibles será determinada por la API de datos. La interfaz debe mostrar el historial según los lotes efectivamente recolectados por el equipo.

## 4.10. RF10. Observabilidad y pruebas de carga

La API de inferencia debe exponer métricas para Prometheus. Como mínimo se espera observar:

- Cantidad de peticiones recibidas.
- Latencia de inferencia.
- Tasa de errores.

- Estado o versión del modelo cargado.
- Consumo de recursos del contenedor, cuando sea posible desde Kubernetes.

Se debe desplegar Locust y ejecutar pruebas de carga contra la API de inferencia. El estudiante debe mostrar en Grafana el efecto de la prueba de carga sobre la API, por ejemplo aumento de peticiones, variación de latencia, errores y comportamiento del contenedor.

## 5. Diseño esperado de DAGs en Airflow

El DAG principal debe tener múltiples tareas y bifurcaciones. La siguiente estructura es una referencia mínima; los estudiantes pueden ampliarla si justifican técnicamente su diseño.

1. **start:** inicio de la ejecución.
2. **fetch\_batch\_from\_api:** consume un lote desde la API externa.
3. **store\_raw\_batch:** almacena la respuesta original en RAW DATA.
4. **validate\_schema:** valida columnas, tipos y estructura del lote.
5. **validate\_data\_quality:** evalúa calidad, nulos, duplicados, rangos y consistencia.
6. **detect\_new\_categories:** identifica categorías nuevas y define cómo incorporarlas sin romper el pipeline.
7. **detect\_data\_drift:** compara la distribución del lote con una referencia histórica.
8. **preprocess\_data:** transforma el lote y actualiza CLEAN DATA.
9. **decide\_training:** bifurca el flujo entre entrenar o no entrenar.
10. **skip\_training:** registra la razón por la cual no se entrenó.
11. **train\_candidate\_model:** entrena un modelo candidato.
12. **evaluate\_candidate\_model:** calcula métricas y genera artefactos.
13. **register\_candidate\_in\_mlflow:** registra el modelo candidato y sus artefactos.
14. **compare\_with\_production:** compara el candidato contra el modelo productivo actual.
15. **decide\_promotion:** bifurca entre promover o rechazar el candidato.
16. **promote\_model:** actualiza alias, stage o tag productivo en MLflow.
17. **reject\_model:** registra la razón del rechazo.
18. **notify\_or\_log\_result:** actualiza la tabla de historial visible por Streamlit.
19. **end:** fin de la ejecución.

El DAG debe ser resistente a fallos. Como mínimo, se espera el uso de reintentos, manejo de errores de conexión, validaciones antes de escritura, logs claros y estados finales interpretables. No se debe ocultar un error crítico como si fuera una ejecución exitosa.

## 6. Componentes

### 6.1. API Data Source

Este componente entrega los lotes de datos que alimentan el sistema. No es desarrollado por los estudiantes, pero debe ser desplegado o consumido según las instrucciones del docente. La API simula una fuente externa de datos que cambia en el tiempo.

Los estudiantes deben implementar un cliente robusto para consumirla desde Airflow. Este cliente debe manejar timeouts, errores HTTP, respuestas vacías, fin de datos disponibles y reintentos controlados.

## 6.2. Airflow

Airflow es el orquestador principal del flujo de datos y entrenamiento. Debe ejecutarse en Kubernetes y sus DAGs deben estar versionados en Git. El despliegue puede apoyarse en Helm si el equipo lo considera conveniente.

Airflow debe encargarse de recolectar datos, validar, procesar, decidir si se entrena, ejecutar el entrenamiento, registrar en MLflow, comparar modelos y registrar el historial del proceso. Cada tarea debe tener una responsabilidad clara. No es aceptable implementar todo el flujo en una única tarea monolítica.

## 6.3. MLflow

MLflow debe funcionar como servidor central de experimentos, artefactos y registro de modelos. Debe usar una base de datos externa para metadatos y un bucket para artefactos. No se permite SQLite como base de metadatos para este proyecto.

MLflow debe conservar todas las versiones relevantes de modelos. El modelo productivo debe poder identificarse mediante un mecanismo consistente: alias, tag, stage o convención documentada. La API de inferencia debe consultar este mecanismo para cargar el modelo correcto.

## 6.4. Sistema de archivos o bucket

El almacenamiento de artefactos debe ser compatible con MLflow. Se recomienda MinIO por su compatibilidad con S3 y porque puede desplegarse dentro del clúster. El bucket debe conservar modelos, reportes, gráficos, explicaciones, objetos serializados y demás artefactos generados durante la experimentación.

El equipo debe documentar cómo se crean el bucket, credenciales, secretos y políticas de acceso necesarias.

## 6.5. Base de datos de metadatos de MLflow

Debe usarse una base de datos relacional para almacenar metadatos de MLflow. Se recomienda PostgreSQL. Esta base no debe mezclarse conceptualmente con RAW DATA ni CLEAN DATA, aunque el equipo puede usar el mismo motor de base de datos con bases o esquemas separados si lo justifica y configura correctamente.

## 6.6. Base de datos RAW DATA

RAW DATA almacena los lotes tal como llegan desde la API y los eventos de inferencia generados por FastAPI. Debe permitir reconstruir qué datos estaban disponibles en cada ejecución del DAG. También debe guardar metadatos mínimos de ingestión, como identificador de lote, fecha, estado, hash o identificador de respuesta cuando aplique.

## 6.7. Base de datos CLEAN DATA

CLEAN DATA almacena los datos transformados y listos para entrenamiento. Debe existir una relación trazable entre los registros procesados y los lotes crudos que los originaron. El pipeline de transformación debe ser reproducible y versionado.

## 6.8. FastAPI

FastAPI expone el servicio de inferencia. Debe cargar el modelo productivo desde MLflow y responder predicciones a partir de los datos ingresados por el usuario o por pruebas de carga. Debe registrar cada inferencia y exponer métricas para Prometheus.

La API debe incluir, como mínimo:

- Endpoint de salud, por ejemplo `/health`.
- Endpoint de inferencia, por ejemplo `/predict`.



- Endpoint de métricas, por ejemplo `/metrics`.
- Mecanismo documentado de recarga del modelo desde MLflow.
- Validación de entrada mediante esquemas, por ejemplo con Pydantic.

## 6.9. Streamlit

Streamlit es la interfaz de usuario del proyecto. Debe consumir FastAPI para inferencia y consultar la base de datos o servicio correspondiente para mostrar el historial de entrenamiento. No debe reemplazar a MLflow; su función es presentar al usuario final y al evaluador una vista clara del comportamiento del sistema.

La interfaz debe mostrar explícitamente la versión del modelo usada, la predicción obtenida y el historial de decisiones por lote.

## 6.10. Locust

Locust debe utilizarse para ejecutar pruebas de carga sobre FastAPI. El repositorio debe incluir el archivo de prueba, instrucciones de ejecución y evidencia del comportamiento observado. La prueba debe ser suficientemente clara para mostrar cambios en métricas de Prometheus y Grafana.

## 6.11. Prometheus y Grafana

Prometheus debe recolectar métricas de la API de inferencia. Grafana debe visualizar dichas métricas en al menos un dashboard. Se espera que el dashboard permita evidenciar el impacto de las pruebas de carga. Es recomendable versionar el dashboard mediante ConfigMap u otro mecanismo declarativo.

## 6.12. GitHub Actions

GitHub Actions debe automatizar la integración continua. Como mínimo, debe construir las imágenes propias, etiquetarlas y publicarlas en DockerHub. Se espera que existan workflows para los componentes desarrollados por el equipo, por ejemplo FastAPI, Streamlit y cualquier imagen personalizada de Airflow o tareas de entrenamiento.

El versionamiento de imágenes debe ser claro. Se recomienda publicar etiquetas asociadas al commit SHA y, cuando aplique, una etiqueta estable para ambientes de desarrollo.

## 6.13. DockerHub

DockerHub funciona como registro de imágenes. Kubernetes no debe depender de imágenes construidas localmente. Los manifiestos deben referenciar imágenes publicadas y versionadas.

## 6.14. Kubernetes

Todos los componentes deben estar desplegados en Kubernetes. Los estudiantes deben definir manifiestos o charts que incluyan, según corresponda, `Deployment`, `StatefulSet`, `Service`, `Ingress`, `ConfigMap`, `Secret`, `PersistentVolumeClaim`, probes y recursos.

Cada componente debe declarar:

- `resources.requests.cpu`
- `resources.requests.memory`
- `resources.limits.cpu`
- `resources.limits.memory`
- `readinessProbe`, cuando aplique.
- `livenessProbe`, cuando aplique.

## 6.15. Argo CD

Argo CD debe sincronizar el estado deseado del repositorio Git con el clúster de Kubernetes. El equipo debe demostrar que los cambios en manifiestos o versiones de imagen se reflejan mediante sincronización de Argo CD. No se considera suficiente aplicar manifiestos manualmente con `kubectl apply` como mecanismo principal de despliegue.

## 7. Requerimientos no funcionales

- **Reproducibilidad:** debe ser posible reconstruir qué datos, código, parámetros y artefactos generaron cada modelo.
- **Trazabilidad:** cada lote debe tener un historial claro desde su ingestión hasta la decisión de entrenamiento y posible promoción.
- **Resiliencia:** el sistema debe manejar errores esperables sin perder el estado del proceso.
- **Escalabilidad mínima:** los servicios deben poder ejecutarse con límites de recursos definidos y comportarse de forma observable bajo carga.
- **Mantenibilidad:** el repositorio debe estar organizado por componentes y contener instrucciones claras.
- **Seguridad básica:** credenciales, tokens y contraseñas no deben quedar quemados en el código fuente. Deben usarse Secrets o mecanismos equivalentes.
- **Observabilidad:** la API debe exponer métricas útiles y el equipo debe demostrar su visualización en Grafana.

## 8. Entregables

1. Repositorio público o accesible para el docente con todo el código fuente.
2. DAGs de Airflow funcionales y documentados.
3. Workflows de GitHub Actions para construir y publicar imágenes en DockerHub.
4. Manifiestos o charts de Kubernetes versionados en Git.
5. Aplicación de Argo CD configurada para desplegar el sistema.
6. MLflow funcional con backend de metadatos y almacenamiento de artefactos.
7. Bases o esquemas RAW DATA y CLEAN DATA con evidencia de uso.
8. FastAPI funcional consumiendo el modelo productivo desde MLflow.
9. Streamlit con inferencia e historial de entrenamiento/despliegue.
10. Prometheus y Grafana recolectando y visualizando métricas de la API.
11. Locust configurado y evidencia de prueba de carga.
12. Documentación técnica del proyecto, decisiones de diseño, criterios de entrenamiento y reglas de promoción.
13. Video de sustentación publicado en YouTube con duración máxima de 10 minutos.

## 9. Criterios mínimos de evaluación técnica

| <b>Criterio</b>           | <b>Evidencia esperada</b>                                                                                     |
|---------------------------|---------------------------------------------------------------------------------------------------------------|
| Orquestación              | DAGs con múltiples tareas, bifurcaciones y manejo de errores.                                                 |
| Datos                     | Separación entre RAW DATA y CLEAN DATA, trazabilidad por lote y registro de inferencias.                      |
| Decisión de entrenamiento | Reglas explícitas basadas en datos, cambios de distribución, calidad, volumen, nuevas categorías o desempeño. |
| MLflow                    | Registro completo de experimentos, métricas, artefactos y modelos versionados.                                |
| Promoción de modelos      | Comparación contra modelo productivo y promoción condicionada por métricas.                                   |
| Inferencia                | API que carga el modelo desde MLflow y puede actualizarlo sin redesplicue.                                    |
| Streamlit                 | Interfaz de inferencia e historial claro de lotes, decisiones y resultados.                                   |
| CI/CD                     | Imágenes construidas por GitHub Actions y publicadas en DockerHub.                                            |
| GitOps                    | Despliegue gestionado por Argo CD desde manifiestos versionados.                                              |
| Kubernetes                | Recursos, probes, servicios, secretos, persistencia y despliegues correctamente definidos.                    |
| Observabilidad            | Métricas de API visibles en Grafana y evidencia de prueba de carga con Locust.                                |
| Documentación             | Explicación técnica clara, decisiones justificadas y evidencia reproducible.                                  |

## 10. Video de sustentación

Como sustentación, cada equipo debe entregar un video publicado en YouTube con duración máxima de 10 minutos. El video debe mostrar, como mínimo:

- Organización del repositorio.
- Arquitectura implementada y comunicación entre componentes.
- Ejecución o evidencia de los workflows de GitHub Actions.
- Despliegue del sistema mediante Argo CD.
- Ejecución del DAG de Airflow y explicación de sus bifurcaciones.
- Registro de experimentos y modelos en MLflow.
- Caso donde se entrena y se promueve un modelo.
- Caso donde se entrena pero no se promueve, o donde no se entrena por decisión técnica.
- Uso de FastAPI y Streamlit para inferencia.
- Historial de entrenamiento visible en Streamlit.
- Prueba de carga con Locust y efecto observado en Grafana.

## 11. Consideraciones críticas

Este proyecto no se evalúa únicamente por lograr una predicción. Se evalúa la capacidad de construir un sistema MLOps coherente, automatizado, observable y técnicamente defendible. Un modelo con buen desempeño pero sin trazabilidad, sin control de versiones, sin justificación de reentrenamiento o sin despliegue reproducible no cumple el objetivo del nivel.

Los estudiantes deben evitar soluciones que dependan de acciones manuales no documentadas. En particular, no se debe actualizar el modelo de la API editando código, copiando archivos manualmente al contenedor o reconstruyendo la imagen de inferencia por cada nuevo modelo. La fuente de verdad del modelo productivo debe ser MLflow.